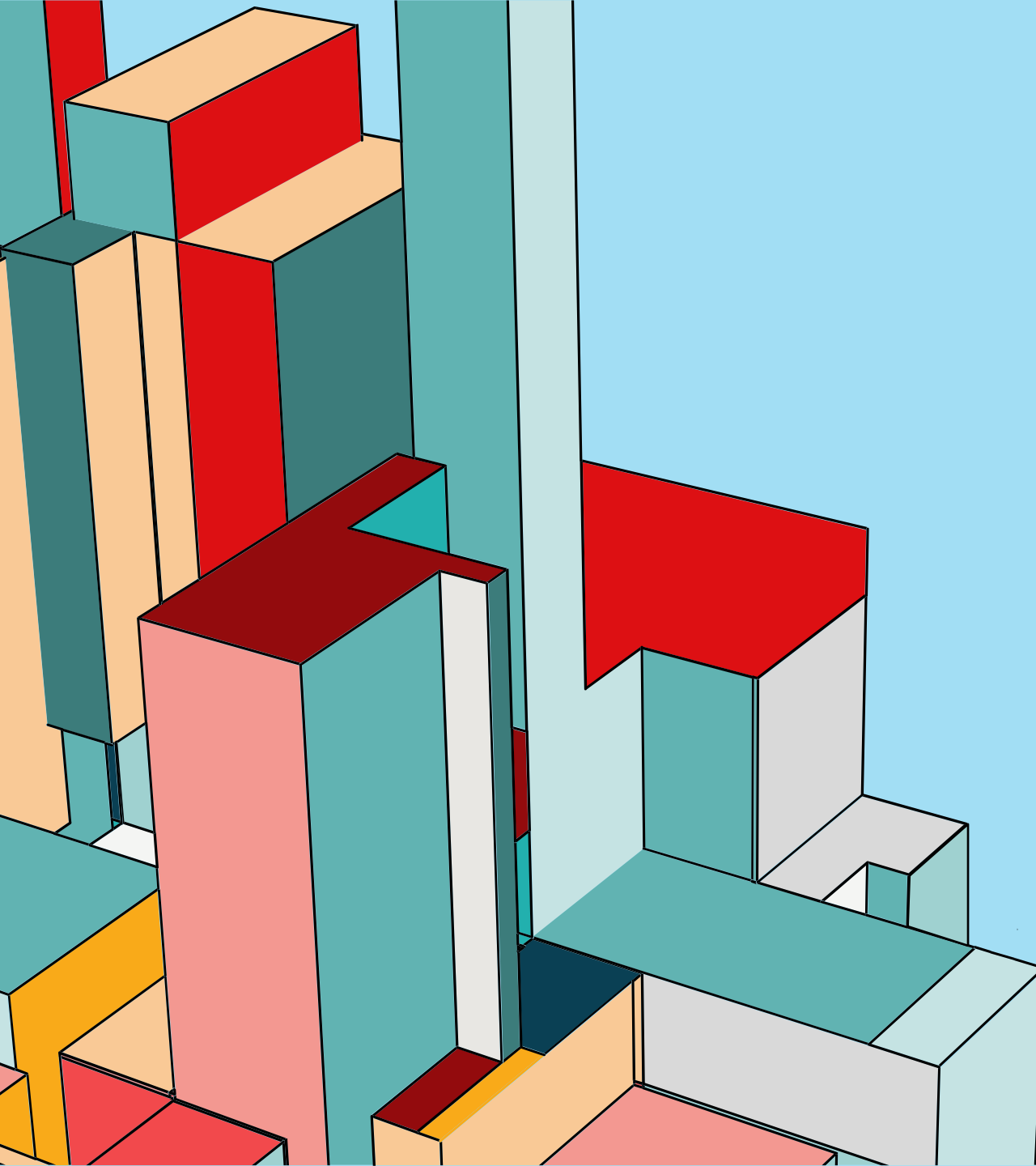


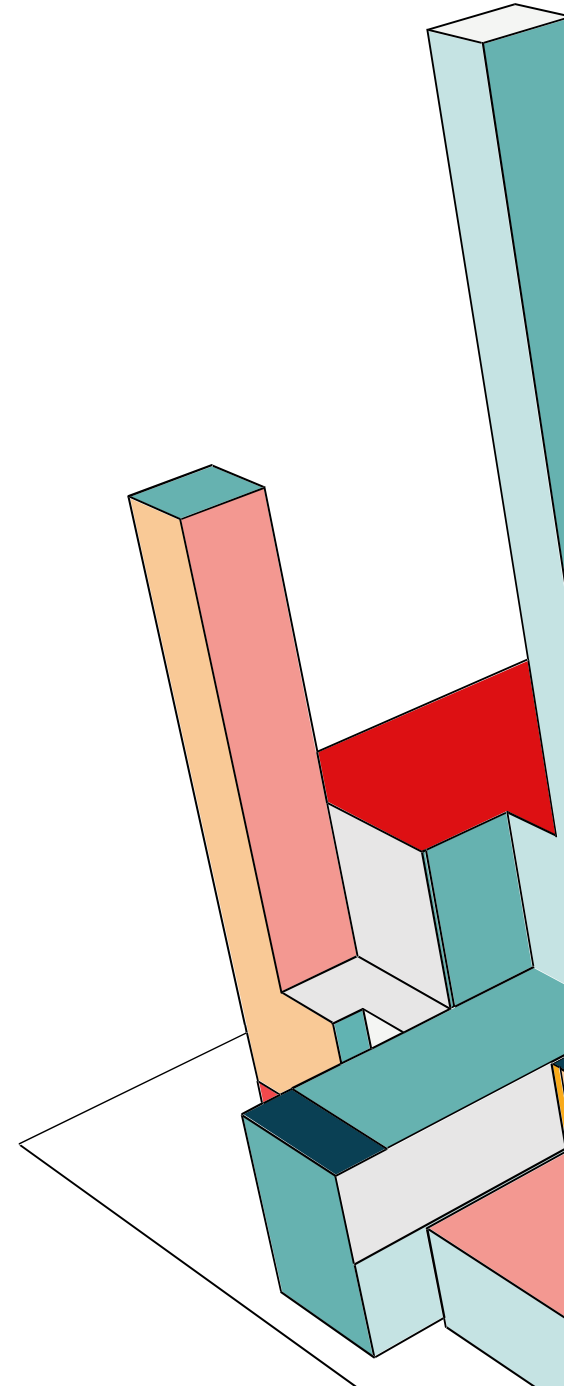


LECTURE 5 **MORE INDUCTIVE ALGORITHMS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Exam 1 is in two weeks, on Wed. 2/11.
 - Details in future classes.
 - Please reach out with accommodations ASAP.
- No office hours for Dr. DiTursi on Tues. 2/3.
 - Email responses may also be sketchy next week.
- Reminder: If you did not finish the lab activities yesterday, please get checked off at any office hours before next week's lab. (DO NOT try to get checked off at the start of next lab period.)



STICKING POINTS FROM LAB

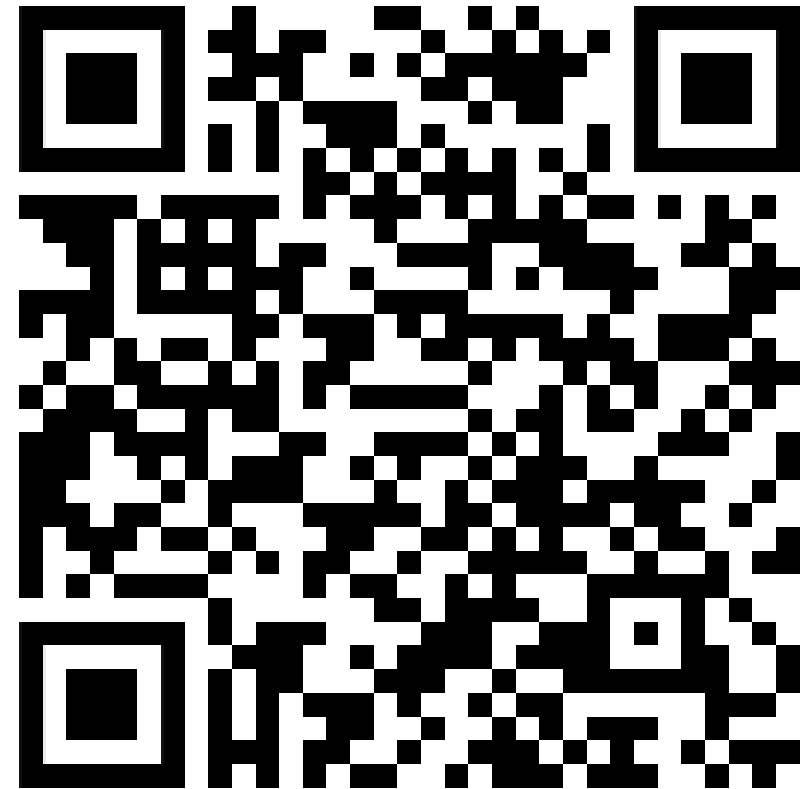
BFS & DFS trees

Topological sort

Broken out in a separate slide deck.

SUBMITTY POLL #1

Which of the given choices is a valid topological sort?



TODAY'S TASKS

- Generating combinatorial objects
- Constant-factor decrease algorithms
 - Fake coin puzzle
 - Binary search
 - Russian peasant multiplication

COMBINATORIAL OBJECTS

By *combinatorial* objects, we mean “the set of all _____ related to some set S of size n ”, where the thing in the blank has at least 2^n elements.

- Most commonly, these are subsets, permutations, or combinations. We’ll focus on the first two.

Perforce, none of these algorithms are tractable on large S , because the number of items we are asked to generate explodes.

Nevertheless, we can work towards generating them as efficiently as possible.

GENERATING PERMUTATIONS

Consider a sorted array containing n distinct integers.

Do now: Write an algorithm for printing all permutations of the array. (You should call **print(A)** to output each permutation.)

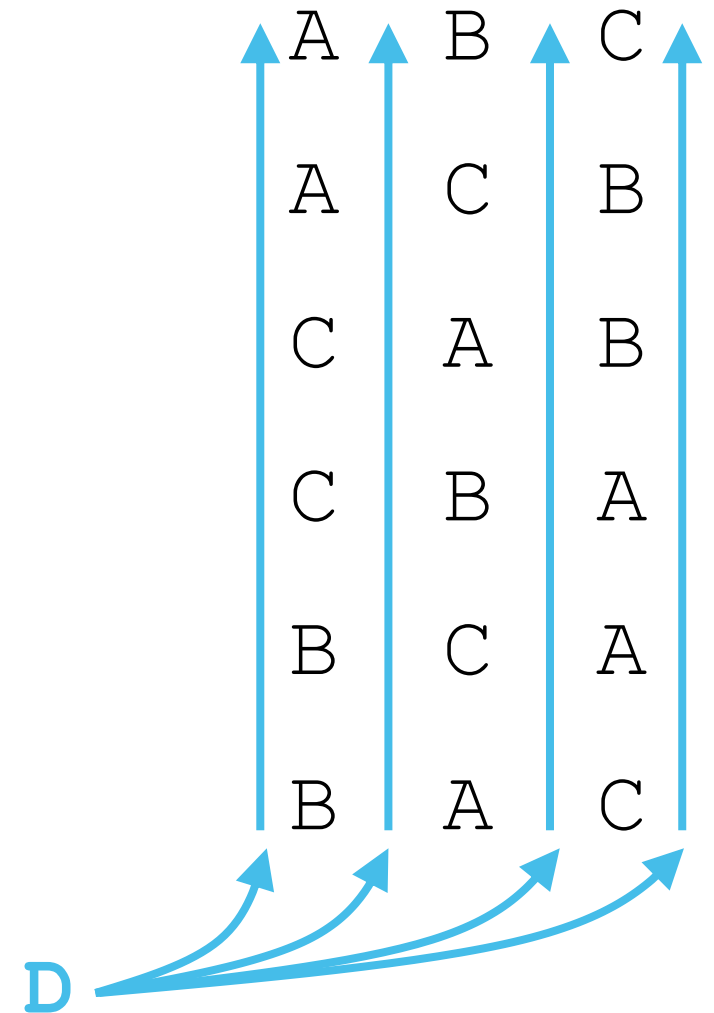
Obviously, every algorithm that accomplishes this requires $\Omega(n!)$ time, as we'll need to make $n!$ calls to **print**.

Every algorithm will also need to swap elements around. An optimal algorithm will manage things so that every swap produces a new permutation, so that we need only $n! - 1$ total swaps for the whole process.

INDUCTIVELY GENERATING PERMUTATIONS

Let's try to think about this inductively.
Assume that we have a process for
producing all permutations of $n - 1$ items.
How do we modify that process to handle
one more element?

The key idea is that our new element must
appear in every possible position in every
single one of those partitions.

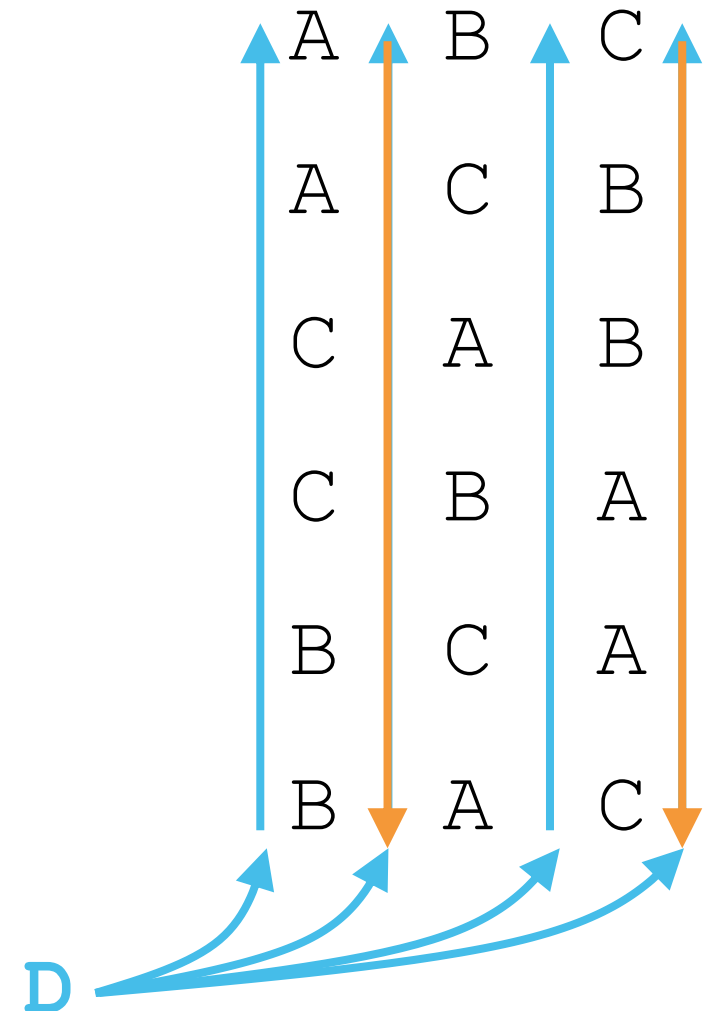


EFFICIENTLY GENERATING PERMUTATIONS

So what order should we generate these permutations in to do so efficiently, with a minimal number of swaps required?

Again, we can work inductively. Observe that in our example, the smaller case with just ABC is already in an efficient order.

So we can keep D in place and move from row to row. The only trick is that when we finish one column, we must reverse direction for the next column.



PRACTICAL MATTERS

We know this is going to take a long time, but we can't possibly take up $O(n!)$ space as well - which is what will happen if we try to store the entire list of size $n - 1$ permutations.

We need a "bottom-up" approach to generating these permutations that just relies on swapping elements.

THE JOHNSON-TROTTER ALGORITHM

Define a *mobile* element here as one that points at a smaller neighbor.

We assume our input is the integers 1 to n , since we can always use these as subscripts into our actual array later.

```
def Johnson-Trotter(A):
    Set the direction of all elements to left
    print(A)
    while A has any mobile elements:
        Let  $k$  be the largest mobile element
        Swap  $k$  with what it's pointing at
        print(A)
        Change the direction of all elements  $> k$ 
```

$\overleftarrow{1} \overleftarrow{2} \overleftarrow{3} \overleftarrow{4}$	$\overrightarrow{4} \overrightarrow{3} \overleftarrow{2} \overleftarrow{1}$
$\overleftarrow{1} \overleftarrow{2} \overleftarrow{4} \overleftarrow{3}$	$\overrightarrow{3} \overrightarrow{4} \overleftarrow{2} \overleftarrow{1}$
$\overleftarrow{1} \overleftarrow{4} \overleftarrow{2} \overleftarrow{3}$	$\overrightarrow{3} \overleftarrow{2} \overrightarrow{4} \overleftarrow{1}$
$\overleftarrow{4} \overleftarrow{1} \overleftarrow{2} \overleftarrow{3}$	$\overrightarrow{3} \overleftarrow{2} \overleftarrow{1} \overrightarrow{4}$
$\overrightarrow{4} \overleftarrow{1} \overleftarrow{3} \overleftarrow{2}$	$\overleftarrow{2} \overrightarrow{3} \overleftarrow{1} \overleftarrow{4}$
$\overleftarrow{1} \overrightarrow{4} \overleftarrow{3} \overleftarrow{2}$	$\overleftarrow{2} \overrightarrow{3} \overleftarrow{4} \overleftarrow{1}$
$\overleftarrow{1} \overleftarrow{3} \overrightarrow{4} \overleftarrow{2}$	$\overleftarrow{2} \overleftarrow{4} \overrightarrow{3} \overleftarrow{1}$
$\overleftarrow{1} \overleftarrow{3} \overleftarrow{2} \overrightarrow{4}$	$\overleftarrow{4} \overleftarrow{2} \overrightarrow{3} \overleftarrow{1}$
$\overleftarrow{3} \overleftarrow{1} \overleftarrow{2} \overleftarrow{4}$	$\overrightarrow{4} \overleftarrow{2} \overleftarrow{1} \overrightarrow{3}$
$\overleftarrow{3} \overleftarrow{1} \overleftarrow{4} \overleftarrow{2}$	$\overleftarrow{2} \overrightarrow{4} \overleftarrow{1} \overrightarrow{3}$
$\overleftarrow{3} \overleftarrow{4} \overleftarrow{1} \overleftarrow{2}$	$\overleftarrow{2} \overleftarrow{1} \overrightarrow{4} \overrightarrow{3}$
$\overleftarrow{4} \overleftarrow{3} \overleftarrow{1} \overleftarrow{2}$	$\overleftarrow{2} \overleftarrow{1} \overrightarrow{3} \overrightarrow{4}$

GENERATING SUBSETS

Given a set S of n objects (call them $a_1, a_2, \dots, a_n$), how do we generate every possible subset of S ?

Think inductively!

- Assume we can generate all subsets of $\{a_1, \dots, a_{n-1}\}$.
- Make two copies of each: one with a_n , one without.

This translates very easily to a bottom-up approach:

$n = 0$: $\emptyset$

$n = 1$: $\emptyset \quad \{a_1\}$

$n = 2$: $\emptyset \quad \{a_1\} \quad \{a_2\} \quad \{a_1, a_2\}$

$n = 3$: $\emptyset \quad \{a_1\} \quad \{a_2\} \quad \{a_1, a_2\} \quad \{a_3\} \quad \{a_1, a_3\} \quad \{a_2, a_3\} \quad \{a_1, a_2, a_3\}$

SUBSETS AND BITSTRINGS

How do we represent subsets?

One very compact way of doing so is by using a bitstring of length n – there are 2^n strings and 2^n subsets, so we can put them into 1:1 correspondence.

Further, there's a very intuitive mapping: each bit b_i is 1 when element a_i is in the subset and 0 when it is not.

$$0101 \leftrightarrow \{a_2, a_4\} \quad 1110 \leftrightarrow \{a_1, a_2, a_3\} \quad 0000 \leftrightarrow \emptyset \quad \text{etc.}$$

Thus, one way of generating subsets is simply counting from 0 to $n - 1$ in binary. But this can result in big changes in consecutive subsets (e.g. going from 011 ... 111 to 100 ... 000).

EFFICIENTLY GENERATING SUBSETS

As with permutations, we'd like to generate the subsets in such a way that each one is minimally changed from the previous one – this would be the most efficient process.

A “change” in this instance is either adding a single element to a subset or subtracting one from it.

This corresponds to changing a single bit from 0 to 1 or vice versa.

Thus, our problem reduces to: Is there a way to arrange the 2^n bitstrings of length n so that every pair of consecutive strings differs by exactly one bit?

GRAY CODE

Frank Gray, a researcher at Bell Labs, developed this in the 1940s to aid in digital signal transmission.

Of course, this is an instance of Stigler's Law. Émile Baudot first published this back in the 1870s while working on telegraphs.

The *binary reflected Gray code* can be generated in an inductive, recursive fashion as follows:

```
def GrayCode(n):  
    if n == 1 return {0,1}  
    else:  
        L = GrayCode(n-1)  
        L1 = '0' + L    # append a 0 to each string in L  
        L2 = '1' + reverse(L) # reverse the order of L first  
        return L1 followed by L2
```

GRAY CODE EXAMPLES

$n = 1$: 0, 1

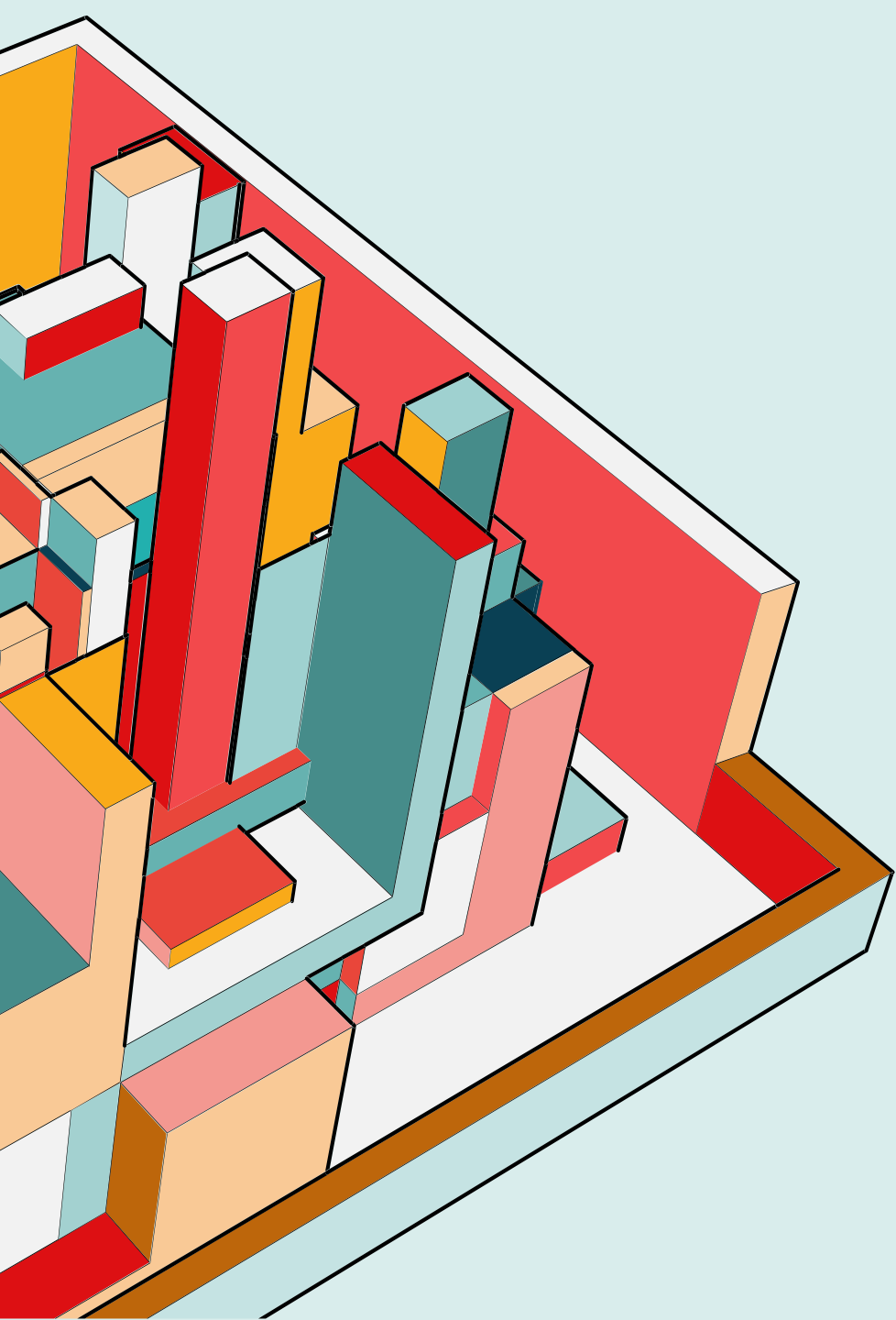
$n = 2$: 00, 01, 11, 10

Do the next two sizes now!

$n = 3$: 000, 001, 011, 010, 110, 111, 101, 100

$n = 4$: 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100,
1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000

Observe that, like the Johnson-Trotter approach to permutations, the resulting order is cyclical: one additional minimal change takes us from the last item back to the first.



CONSTANT-FACTOR DECREASE ALGORITHMS

THE COUNTERFEIT COIN PUZZLE

A classic puzzle goes like this:

You have a pile of 100 gold coins. One of the coins is counterfeit and weighs less than the other 99 coins, which are all identical. How can you identify the counterfeit coin in as few weighings as possible?



SUBMITTY POLL #2

What is the smallest number of weighings that can guarantee finding the counterfeit coin?



THE COUNTERFEIT COIN PUZZLE



A classic puzzle goes like this:

You have a pile of 100 gold coins. One of the coins is counterfeit and weighs less than the other 99 coins, which are all identical. How can you identify the counterfeit coin in as few weighings as possible?

≤ 99 weighings: Put one coin in the left pan, and cycle through the others in the right pan until you find a lighter coin.

≤ 50 weighings: Cycle through pairs of coins.

≤ 7 weighings: Put 50 coins in each pan. Split the lighter side in half and weigh the halves against each other.

≤ 5 weighings: Split into three groups. Balance the two equal-size ones; if they match, the counterfeit is in the third pile.

DECREASE BY A CONSTANT FACTOR

The latter two approaches resemble the inductive algorithms that we've seen recently. The key difference is that instead of subtracting a fixed amount (usually 1) from the problem size at each step, now we're dividing the problem size by a fixed amount at each step.

$$T(n) = T(n - 1) + \textit{stuff}$$

vs.

$$T(n) = T(n/2) + \textit{stuff}$$

The best-known example of this approach is, of course, binary search.

BINARY SEARCH

Binary search is essentially playing the “high-low” guessing game you may have encountered as a kid: We start with the middle value, find out if our target is lower or higher than that value, and then ignore the half of the list that the target **cannot** be in. Then we repeat the process until we find the target or have proven it can’t possibly be in the list.

Note that obtaining the “high-low” information cheaply requires that the list be sorted.

3	14	27	31	39	42	55	70	74	81	85	93	98
---	----	----	----	----	----	----	----	----	----	----	----	----

BINARY SEARCH PSUEUDOCODE

```
def binarySearch(A,n,target):  
    left = 0; right = n-1  
    while left <= right:  
        mid = (left + right) // 2    # integer division  
        if A[mid] == target return mid  
        else:  
            if target < A[mid]:  
                right = mid - 1  
            else:  
                left = mid + 1  
    return -1    # didn't find it
```

RUSSIAN PEASANT MULTIPLICATION

Suggest a pair of three-digit numbers.

Now, multiply them by hand. Go.

RUSSIAN PEASANT MULTIPLICATION EXAMPLE

624	319		
312	638		5104
156	1276		10208
78	2552		20416
39	5104	5104	+ 163328
19	10208	10208	-----
9	20416	20416	199056
4	40832		
2	81664		
1	163328	163328	

RUSSIAN PEASANT MULTIPLICATION

The algorithm rests on two equations:

$$n \cdot m = \begin{cases} (n/2) \cdot 2m & \text{when } n \text{ is even} \\ ((n-1)/2) \cdot 2m + m & \text{when } n \text{ is odd} \end{cases}$$

Since the only operations are doubling, halving, and adding, it might be considered an "easier" approach to multiplying by, say, Russian peasants. (The name comes from Western visitors observing it in use in the 1800s; however, the algorithm was known even in ancient Egypt.)

In binary, doubling/halving are just bit shifts, which makes for VERY fast hardware implementations.

BEFORE YOU LEAVE

